

```

#include <Arduino.h>
#line 1
"C:\\Users\\prade\\Documents\\Arduino\\sketch_oct2b_part1\\sketch_oct2b_part1.ino"

//Pin definitions

// Line sensors
const int PIN_LINE_LEFT   = A0;
const int PIN_LINE_CENTER = A1;
const int PIN_LINE_RIGHT  = A2;

// Ultrasonic sensor
const int PIN_TRIG = 8;
const int PIN_ECHO = 9;

// Motors via ULN2001
const int PIN_MOTOR_LEFT  = 5;
const int PIN_MOTOR_RIGHT = 6;

// State indicator LEDs
const int PIN_LED_FOLLOW = 10;
const int PIN_LED_AVOID  = 11;
const int PIN_LED_STOP   = 12;

//Robot state machine

enum RobotState {
    STATE_FOLLOW = 0,
    STATE_AVOID  = 1,
    STATE_STOP   = 2
};

RobotState currentState = STATE_FOLLOW;

//Timing and scheduler variable

unsigned long lastLineRead      = 0;
unsigned long lastUltraRead     = 0;
unsigned long lastControlUpdate = 0;
unsigned long lastDebugPrint    = 0;

const unsigned long PERIOD_LINE_READ      = 10;    // ms
const unsigned long PERIOD_ULTRA_READ     = 60;    // ms
const unsigned long PERIOD_CONTROL_UPDATE = 10;    // ms
const unsigned long PERIOD_DEBUG_PRINT    = 500;   // ms

//Line following variables

int  lineError = 0;          // -1 means left, 0 means center and +1 means right
bool lineLost  = false;     // true if all sensors see white

```

```

unsigned long lineLostSince = 0;           // time when line was first lost
const unsigned long LINE_LOST_TIMEOUT = 800; // ms before we go to STOP

// Simple proportional control
const int BASE_SPEED = 180; // 0..255
const float Kp = 80.0; // proportional gain

int leftSpeed = 0; // 0..255
int rightSpeed = 0; // 0..255

//Obstacle / avoidance variables

float obstacleDistanceCm = 999.0;
const float OBSTACLE_THRESHOLD_CM = 18.0; // obstacle threshold

// Avoidance sequence
int avoidStep = 0;
unsigned long avoidStepStart = 0;
unsigned long avoidStart = 0;
const unsigned long AVOID_TOTAL_TIMEOUT = 6000; // safety timeout for avoid state
(ms)

// SETUP
#line 89
"C:\\Users\\prade\\Documents\\Arduino\\sketch_oct2b_part1\\sketch_oct2b_part1.ino"
void setup();
#line 119
"C:\\Users\\prade\\Documents\\Arduino\\sketch_oct2b_part1\\sketch_oct2b_part1.ino"
void loop();
#line 153
"C:\\Users\\prade\\Documents\\Arduino\\sketch_oct2b_part1\\sketch_oct2b_part1.ino"
void readLineSensors();
#line 178
"C:\\Users\\prade\\Documents\\Arduino\\sketch_oct2b_part1\\sketch_oct2b_part1.ino"
void readUltrasonic();
#line 201
"C:\\Users\\prade\\Documents\\Arduino\\sketch_oct2b_part1\\sketch_oct2b_part1.ino"
void updateFSM(unsigned long now);
#line 257
"C:\\Users\\prade\\Documents\\Arduino\\sketch_oct2b_part1\\sketch_oct2b_part1.ino"
void runControl(unsigned long now);
#line 275
"C:\\Users\\prade\\Documents\\Arduino\\sketch_oct2b_part1\\sketch_oct2b_part1.ino"
void followControl();
#line 312
"C:\\Users\\prade\\Documents\\Arduino\\sketch_oct2b_part1\\sketch_oct2b_part1.ino"
void startAvoidSequence(unsigned long now);
#line 317
"C:\\Users\\prade\\Documents\\Arduino\\sketch_oct2b_part1\\sketch_oct2b_part1.ino"
void avoidControl(unsigned long now);

```

```

#line 366
"C:\\Users\\prade\\Documents\\Arduino\\sketch_oct2b_part1\\sketch_oct2b_part1.ino"
void setMotors(int left, int right);
#line 375
"C:\\Users\\prade\\Documents\\Arduino\\sketch_oct2b_part1\\sketch_oct2b_part1.ino"
void updateStateLEDs();
#line 385
"C:\\Users\\prade\\Documents\\Arduino\\sketch_oct2b_part1\\sketch_oct2b_part1.ino"
void debugPrint();
#line 89
"C:\\Users\\prade\\Documents\\Arduino\\sketch_oct2b_part1\\sketch_oct2b_part1.ino"
void setup() {
    // Line sensors as digital inputs
    pinMode(PIN_LINE_LEFT, INPUT);
    pinMode(PIN_LINE_CENTER, INPUT);
    pinMode(PIN_LINE_RIGHT, INPUT);

    // Ultrasonic pins
    pinMode(PIN_TRIG, OUTPUT);
    pinMode(PIN_ECHO, INPUT);

    // Motor control pins
    pinMode(PIN_MOTOR_LEFT, OUTPUT);
    pinMode(PIN_MOTOR_RIGHT, OUTPUT);

    // State LEDs
    pinMode(PIN_LED_FOLLOW, OUTPUT);
    pinMode(PIN_LED_AVOID, OUTPUT);
    pinMode(PIN_LED_STOP, OUTPUT);

    // Start with everything off
    setMotors(0, 0);
    updateStateLEDs();

    Serial.begin(9600);
    Serial.println("Line robot starting...");
}
//MAIN LOOP (cooperative scheduler)

void loop() {
    unsigned long now = millis();

    // 1) Read line sensors periodically
    if (now - lastLineRead >= PERIOD_LINE_READ) {
        lastLineRead = now;
        readLineSensors();
    }

    // 2) Read ultrasonic sensor periodically
    if (now - lastUltraRead >= PERIOD_ULTRA_READ) {
        lastUltraRead = now;

```

```

    readUltrasonic();
}

// 3) Run FSM + control periodically
if (now - lastControlUpdate >= PERIOD_CONTROL_UPDATE) {
    lastControlUpdate = now;
    updateFSM(now);
    runControl(now);
}

// 4) Optional debug messages
if (now - lastDebugPrint >= PERIOD_DEBUG_PRINT) {
    lastDebugPrint = now;
    debugPrint();
}
}

// SENSOR READING FUNCTIONS

// Read the 3 line sensors and compute error and lineLost
void readLineSensors() {
    // Switch wiring: ON = 5V = HIGH = on top of line (black)
    int L = digitalRead(PIN_LINE_LEFT);
    int C = digitalRead(PIN_LINE_CENTER);
    int R = digitalRead(PIN_LINE_RIGHT);

    // Line lost = all sensors see white (0)
    if (L == LOW && C == LOW && R == LOW) {
        lineLost = true;
    } else {
        lineLost = false;

        // Map patterns to error values
        // You can tune this mapping depending on your track
        if (L == HIGH && C == LOW && R == LOW) lineError = -1; // Line far left
        else if (L == LOW && C == HIGH && R == LOW) lineError = 0; // Centered
        else if (L == LOW && C == LOW && R == HIGH) lineError = 1; // Line far
right
        else if (L == HIGH && C == HIGH && R == LOW) lineError = -1; // between L &
C
        else if (L == LOW && C == HIGH && R == HIGH) lineError = 1; // between C &
R
        else if (L == HIGH && C == LOW && R == HIGH) lineError = 0; // unusual,
treat as center
        else lineError = 0;
    }
}

// Read distance from HC-SR04 in cm
void readUltrasonic() {

```

```

// Trigger a 10us pulse
digitalWrite(PIN_TRIG, LOW);
delayMicroseconds(2);
digitalWrite(PIN_TRIG, HIGH);
delayMicroseconds(10);
digitalWrite(PIN_TRIG, LOW);

// Measure echo pulse width (max 30ms → ~5 meters)
long duration = pulseIn(PIN_ECHO, HIGH, 30000);

if (duration == 0) {
    // No echo detected
    obstacleDistanceCm = 999.0;
} else {
    obstacleDistanceCm = (duration * 0.0343) / 2.0; // convert to cm
}
}

//STATE MACHINE

void updateFSM(unsigned long now) {
    switch (currentState) {
        case STATE_FOLLOW:
            // Obstacle check
            if (obstacleDistanceCm < OBSTACLE_THRESHOLD_CM) {
                // Enter avoid mode
                currentState = STATE_AVOID;
                avoidStart = now;
                startAvoidSequence(now);
                updateStateLEDs();
                break;
            }

            // Line lost handling
            if (lineLost) {
                if (lineLostSince == 0) {
                    lineLostSince = now; // start timing
                } else if (now - lineLostSince > LINE_LOST_TIMEOUT) {
                    currentState = STATE_STOP;
                    updateStateLEDs();
                }
            } else {
                lineLostSince = 0; // reset timer when back on line
            }
            break;

        case STATE_AVOID:
            // Safety timeout: if avoidance takes too long, stop
            if (now - avoidStart > AVOID_TOTAL_TIMEOUT) {
                currentState = STATE_STOP;
                updateStateLEDs();
            }
    }
}

```

```

    }

    // if obstacle gone and line is visible, return to follow
    if (!lineLost && obstacleDistanceCm >= OBSTACLE_THRESHOLD_CM &&
        (now - avoidStart > 1000)) {
        currentState = STATE_FOLLOW;
        updateStateLEDs();
    }
    break;

case STATE_STOP:
    // if line is visible and no obstacle, go back to FOLLOW
    if (!lineLost && obstacleDistanceCm >= OBSTACLE_THRESHOLD_CM) {
        currentState = STATE_FOLLOW;
        lineLostSince = 0;
        updateStateLEDs();
    }
    break;
}
}

// CONTROL / ACTUATION

void runControl(unsigned long now) {
    switch (currentState) {
        case STATE_FOLLOW:
            followControl();
            break;

        case STATE_AVOID:
            avoidControl(now);
            break;

        case STATE_STOP:
            setMotors(0, 0);
            break;
    }
}

// FOLLOW MODE CONTROL (line following)

void followControl() {
    if (lineLost) {
        // If we just lost the line, slow down / stop
        setMotors(0, 0);
        return;
    }

    // Proportional correction based on lineError
    float correction = Kp * (float)lineError;

```

```

float left  = (float)BASE_SPEED - correction;
float right = (float)BASE_SPEED + correction;

// Clamp to 0..255
if (left < 0)   left = 0;
if (left > 255) left = 255;
if (right < 0)  right = 0;
if (right > 255) right = 255;

leftSpeed  = (int)left;
rightSpeed = (int)right;

setMotors(leftSpeed, rightSpeed);
}

// AVOID MODE CONTROL (simple timed sequence)

// Because we can't reverse motors with ULN2001, we "turn" by running one motor and
// stopping the other.

// Sequence (avoidStep):
// 0: stop for 200 ms
// 1: turn right (left motor ON, right OFF) for 700 ms
// 2: go straight (both ON) for 900 ms
// 3: turn left (right ON, left OFF) for 700 ms
// 4: go straight (both ON) – stay here until FSM moves us back to FOLLOW
//
void startAvoidSequence(unsigned long now) {
    avoidStep = 0;
    avoidStepStart = now;
}

void avoidControl(unsigned long now) {
    switch (avoidStep) {
        case 0: // stop briefly
            setMotors(0, 0);
            if (now - avoidStepStart > 200) {
                avoidStep = 1;
                avoidStepStart = now;
            }
            break;

        case 1: // turn right
            // left motor ON, right motor OFF
            setMotors(BASE_SPEED, 0);
            if (now - avoidStepStart > 700) {
                avoidStep = 2;
                avoidStepStart = now;
            }
            break;
    }
}

```

```

    case 2: // go straight
        setMotors(BASE_SPEED, BASE_SPEED);
        if (now - avoidStepStart > 900) {
            avoidStep = 3;
            avoidStepStart = now;
        }
        break;

    case 3: // turn left
        // left motor OFF, right motor ON
        setMotors(0, BASE_SPEED);
        if (now - avoidStepStart > 700) {
            avoidStep = 4;
            avoidStepStart = now;
        }
        break;

    case 4: // go straight until FSM switches back to FOLLOW
        setMotors(BASE_SPEED, BASE_SPEED);
        // No timing change here; FSM decides when to exit AVOID
        break;
}
}

// LOW-LEVEL MOTOR + LED HELPERS

// Set motor speeds 0..255 using PWM on pins 5 and 6
// Remember: ULN2001 sinks current to ground, motor positives go to battery +
void setMotors(int left, int right) {
    left = constrain(left, 0, 255);
    right = constrain(right, 0, 255);

    analogWrite(PIN_MOTOR_LEFT, left);
    analogWrite(PIN_MOTOR_RIGHT, right);
}

// Update LEDs based on current state
void updateStateLEDs() {
    digitalWrite(PIN_LED_FOLLOW, (currentState == STATE_FOLLOW));
    digitalWrite(PIN_LED_AVOID, (currentState == STATE_AVOID));
    digitalWrite(PIN_LED_STOP, (currentState == STATE_STOP));
}

// DEBUG OUTPUT

void debugPrint() {
    Serial.print("STATE = ");
    if (currentState == STATE_FOLLOW)        Serial.print("FOLLOW");

```



```
else if (currentState == STATE_AVOID) Serial.print("AVOID");
else Serial.print("STOP");

Serial.print(" | lineError=");
Serial.print(lineError);
Serial.print(" | lineLost=");
Serial.print(lineLost ? "YES" : "NO");

Serial.print(" | dist=");
Serial.print(obstacleDistanceCm);
Serial.print(" cm");

Serial.print(" | Lspeed=");
Serial.print(leftSpeed);
Serial.print(" Rspeed=");
Serial.println(rightSpeed);
}
```